



沈阳建筑大学

上机实验报告

课程名称:	编译原理
实验题目:	实验二 语法分析
专业班级:	计算机 2101 班
姓 名:	张清晨
学 号:	2104230414
日 期:	2024.05.04

【实验目的】

上机目的：

通过语法分析上机掌握形式化问题的方法，运用上下文无关文法描述高级程序设计语言的语句结构，设计并实现满足实际需求的语法分析程序。

内容：

- (1) 了解常用的自上而下语法分析方法；
- (2) 掌握根据产生式实现递归下降分析程序的方法；
- (3) 通过设计、编制、调试一个递归下降语法分析程序，实现对算术表达式进行语法检查和结构分析。

【基本原理和上机步骤】

基本原理：

识别程序由一组子程序组成，每个子程序对应一个非终结符。

该子程序根据下一个输入符号（SELECT 集）来确定按照哪一个产生式进行处理，再根据该产生式的右端：

- 每遇到一个终结符，判断当前读入的单词是否与该终结符匹配，若匹配继续读取下一个单词，若不匹配，则进行出错处理；
- 每遇到一个非终结符，则调用相应的子程序。

递归下降分析法的前提：

消除二义性、消除左递归、提取左公因子，判断是否为 LL(1)文法

上机步骤：

自上向下分析过程中，如果带回溯，则分析过程是穷举所有可能的推导，看是否能推导出待检查的符号串。分析速度慢。而无回溯的自上向下分析技术，当选择某非终结符的产生时，可根据输入串的当前符号以及各产生式右部首符号而进行，效率高，且不易出错。

无回溯的自上向下分析技术可用的先决条件是：无左递归和无回溯。

无左递归：既没有直接左递归，也没有间接左递归。

无回溯：对于任一非终结符号 U 的产生式右部 $x_1|x_2|\cdots|x_n$ ，其对应的字的首终结符号两两不相交。

如果一个文法不含回路（形如 $P \Rightarrow^+ P$ 的推导），也不含以 ϵ 为右部的产生式，那么可以通过执行消除文法左递归的算法消除文法的一切左递归（改写后的文法可能含有以 ϵ 为右部的产生式）。

文法的左递归消除算法：

1、将文法 G 的所有非终结符排序为 $U_1, U_2, \dots, U_n$;

2、For ($i=1$; $i++$; $i \geq n$)

{

for $j \rightarrow 1$ to $i-1$

把产生式 $U_i \rightarrow U_j \alpha$ 替换成 $U_i \rightarrow \beta_1 \alpha \mid \beta_2 \alpha \mid \dots \mid \beta_m \alpha$;

其中: $U_j \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$

消除 U_i 产生式中的直接左递归;

}

3、化简改写之后的文法, 删除多余产生式。

文法的直接左递归消除公式:

直接左递归形式: $U \rightarrow UX \mid y$;

其中: $x, y \in (V_N \cup V_T)^*$, y 不以 U 打头。

直接左递归的消除:

$U \rightarrow yU'$

$U' \rightarrow xU' \mid \varepsilon$

直接左递归的一般形式: $U \rightarrow UX_1 \mid UX_2 \mid \dots \mid UX_m \mid y_1 \mid y_2 \mid \dots \mid y_n$;

其中: $x_i \neq \varepsilon$, y_i 都不以 U 打头。

一般形式直接左递归的消除:

$U \rightarrow y_1U' \mid y_2U' \mid \dots \mid y_nU'$

$U' \rightarrow x_1U' \mid x_2U' \mid \dots \mid x_mU' \mid \varepsilon$

回溯的消除的前提是文法不得含有左递归, 可提左因子来消除回溯。

递归下降分析程序的基本架构如下的法:

先定义:

变量: ch : 当前符号;

函数: $READ(ch)$: 读输入串下一符号;

对于每个非终结符号 $U \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ 处理的方法如下:

$P(U)$

{

if $ch \in FIRST(\alpha_1)$ then $P(\alpha_1)$ //处理 α_1 的程序部分。

else if $ch \in FIRST(\alpha_2)$ then $P(\alpha_2)$ //处理 α_2 的程序部分。

...

else if $ch \in FIRST(\alpha_n)$ then $P(\alpha_n)$

```

else if  $ch \in FOLLOW(U)$  then return //处理空产生式情况。
else error
}

```

对于每个右部 $\alpha = x_1x_2\cdots x_n$ 的处理架构如下：

```

P( $\alpha$ )
{
    P( $x_1$ );           //处理  $x_1$  的程序。
    P( $x_2$ );           //处理  $x_2$  的程序。
    ... ..
    P( $x_n$ );           //处理  $x_n$  的程序。
}

```

对于右部中的每个符号 x_i ：

如果 x_i 为终结符号：

```

if(ch= a)
    READ (ch)           //对于终结符，直接将指针前调。
else
    error

```

如果 x_i 为非终结符号，直接调用相应的过程：p (x_i)。

【实验结果】

输入测试字符串 1，运行结果：

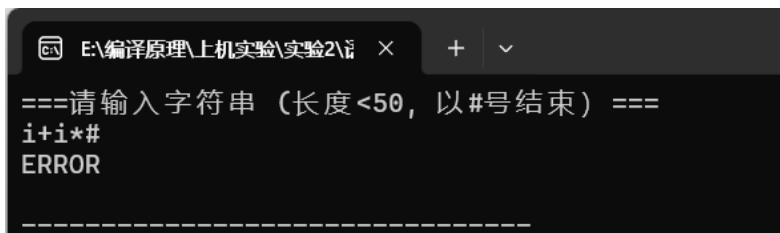
```

E:\编译原理\上机实验\实验2\1 x + v
===请输入字符串（长度<50，以#号结束）===
i+i*i#

====>      输入串分析正确！
推导过程如下：
文法      分析栈      当前分析字符
E          E#         i
E-->TG     TG#         i
T-->FS     FSG#        i
F-->i      SG#         +
S-->ε      G#          +
G-->+TG    TG#         i
T-->FS     FSG#        i
F-->i      SG#         *
S-->*FS    FSG#        i
F-->i      SG#         #
S-->ε      G#          #
G-->ε      #           #

```

输入测试字符串 2，运行结果：



```
E:\编译原理\上机实验\实验2\1 x + v
===请输入字符串（长度<50，以#号结束）===
i+i*#
ERROR
-----
```

【讨论与分析】

功能描述：本次上机所调试的程序是通过确定的自上而下分析法实现的递归下降语法分析程序。对文法中的每个非终结符编写一个函数（或子程序），每个函数（或子程序）的功能是识别由该非终结符所表示的语法成分。不同函数之间有对应的调用关系，这需要根据文法来确定。通过编写的程序，我们可以对输入的字符串进行识别，看其是否属于该文法下的句子。

程序结构描述：函数调用格式、参数含义、返回值描述、函数功能；函数之间的调用关系图。

本次的函数调用格式无特别，均是与示例中的文法非终结符来命名函数名称，函数无参数，无返回值（递归调用）

实验过程记录：在实验的过程中，我有很多次错误，首先我通过自己调试代码的方式解决，如果未能解决，又去上网查询资料，最后都能解决。

实验总结：在本次语法分析实验中，我对语法分析的原理和具体实现方法有了更深刻的认识。通过亲自构建语法分析器，我对编译过程的重要环节有了更全面的理解。实验中遇到的困难和找到的解决策略进一步磨练了我的问题解决技巧和编程能力。例如，当实验中出现错误时，我独立进行了调试并成功修复了这些错误，同时我也积极上网搜索资料以解决遇到的问题。这些经历为我未来在编译器设计和实现方面的努力积累了宝贵的经验。总的来说，这次实验是一次收获丰富的学习体验。

【源程序代码】

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>

using namespace std;

//变量定义
string s, str, stackStr;//s: 输入串、str: 中间变量、stackStr: 模拟栈
int i;
string ch;//当前分析字符
vector<string> v;//字符串类型的向量(文法+分析栈)

//函数声明
void E();           //E-->TG
void G();           //G-->+TG|-TG|ε
void T();           //T-->FS
void S();           //S-->*FS|FS|ε
void F();           //F-->(E)|i
void err();         //提示错误信息
int check();//验证是否已经到栈底
void push(string pre, string value);//将字符串存入输出栈

/**
 * 函数功能: 提示错误信息
 */
void err()
{
    cout << "ERROR" << endl;
    exit(0);
}

/**
 * 函数功能: 将字符串存入输出栈
 */
void push(string pre, string value)
{
    ch += s[i];
    int idx = stackStr.find_first_of(pre[0], 0);//从头开始找到 pre 开始的位置
```

```

    if (value != "ε")//不是空字时
    {
        string value1;
        value1 = value;
        if (value[0] == '+') value1 = "TG";
        if (value[0] == '*') value1 = "FS";
        if (value[0] == '(') value1 = "E";
        if (value[0] == 'i') value1 = "";
        stackStr.replace(idx, 1, value1);//将第一个 pre 的位置替换为 value
    }
    else
    {
        stackStr.erase(idx, 1);//删除从该位置开始的 1 个字符
    }
    v.push_back((pre + value + "," + stackStr));//将对应的表达式和栈的内容存入
    在向量 v 尾部
}
/**
 * 函数功能：验证是否已经到栈底
 */
int check()
{
    if (i >= s.size()) {
        return 1;
    }
    else if (s[i] == '#')
    {
        ch += '#';
        return 1;
    }
    return 0;
}
/**
 * 函数功能：E-->TG
 */
void E()
{
    push("E-->", "TG");
    T();
}

```

```

    G();
}
/**
 * 函数功能:  $G \rightarrow +TG \mid -TG \mid \varepsilon$ 
 */
void G() {
    if (s[i] == '+')
    {
        str = s[i];
        str += "TG";
        i++;
        push("G-->", str);
        T();
        G();
    }
    else if (s[i] == '-')
    {
        str = s[i];
        str += "TG";
        i++;
        push("G-->", str);
        T();
        G();
    }
    else
    {
        push("G-->", "ε");
    }
}
/**
 * 函数功能:  $T \rightarrow FS$ 
 */
void T()
{
    push("T-->", "FS");
    F();
    S();
}
/**

```



```

*  函数功能: S-->*FS|FS| ε
*/
void S() {
    if (s[i] == '*')
    {
        str = s[i];
        str += "FS";
        i++;
        push("S-->", str);
        F();
        S();
    }
    else if (s[i] == '/')
    {
        str = s[i];
        str += "FS";
        i++;
        push("S-->", str);
        F();
        S();
    }
    else
    {
        push("S-->", "ε");
    }
}
/**
*  函数功能: F-->(E)i
*/
void F() {
    if (s[i] == '(')
    {
        i++;
        push("F-->", "(E)");
        E();
        if (s[i] == ')')
        {
            i++;
        }
    }
}

```

```

    else
    {
        err();
    }
}
else if (s[i] == 'i')
{
    i++;
    push("F-->", "i");
}
else
{
    err();
}
}
/**
 *  函数功能： 主函数
 */
int main() {
//                                     cout                                     <<
"===== " << endl;
//      cout << "=== 递归下降分析 ===" << endl;
//                                     cout                                     <<
"===== " << endl;
cout << "===请输入字符串（长度<50, 以#号结束）===" << endl;
while (cin >> s) //输入要分析的字符串
{
    v.clear();
    i = 0;
    stackStr = "E#";//初始化栈
    E();
    if (check())
    {
        cout << endl << "=====>\t\t 输入串分析正确！ " << endl;
        cout << "推导过程如下： " << endl;
        cout << "文法\t\t 分析栈\t\t 当前分析字符\n";
        cout << "E          \t\tE#\t\t" << s[0] << endl;//初始栈的内容
        int i;
        for (i = 0; i < v.size(); i++)

```

```

        {
            cout << v[i].substr(0, v[i].find_first_of(",", 0)) << "\t";
            cout << setiosflags(ios::right) << setw(10) <<
v[i].substr(v[i].find_first_of(",", 0) + 1) << "\t\t";
            cout << ch[i] << endl;
        }
    }
    else
        cout << "==>\t 输入串不符合该文法 " << endl;
}

return 0;
}

```